

# CBC Teachers' Toolkit - Project Process Report

## 1. Project Context and Problem Definition

This project was built to solve a practical challenge in the Ndejje University community context: student-teachers and teachers need fast, reliable, and curriculum-aligned teaching support that works in low-connectivity environments.

The application targets CBC teaching support by offering:

- Offline topic resources
- Notes and favorites
- Scheme of Work builder
- Optional online updates from official sources (NCDC links)

Why this step was necessary:

- The capstone requires a real community-relevant problem, not a generic demo.
- Defining a clear user problem early guided all architecture and feature choices.

## 2. Requirements Analysis and Scope Definition

The implementation was aligned with the capstone brief:

- Kotlin + Jetpack Compose
- MVVM architecture
- LazyColumn/LazyRow dynamic lists
- Navigation across multiple screens
- Room persistence
- Unit/integration testing
- MainActivity as entry point only

Scope decisions:

- Keep app offline-first
- Use copyright-safe linking to NCDC resources instead of bundling copied PDFs
- Add an update button for when internet is available

Why this step was necessary:

- It ensured grading rubric compliance and prevented scope drift under tight deadlines.

## 3. Architecture Design

The app follows a layered design:

- UI layer: Compose screens
- State layer: `SubjectViewModel`
- Data layer: `TopicRepository`
- Persistence: Room database with DAO + entities

Key entities:

- `TopicEntity`
- `FavoriteEntity`
- `NoteEntity`
- `TeachingResourceEntity`
- `SchemeOfWorkEntity`

Why this step was necessary:

- Separating concerns improves maintainability, testability, and easier feature growth.
- MVVM and repository patterns are explicitly expected in the project brief.

## 4. Navigation and Screen Planning

Implemented screens:

- Splash
- Home
- Subjects
- Resource Detail
- Updates
- Library
- About
- Scheme Builder

Why this step was necessary:

- Navigation Compose provides a clear user flow and satisfies the "3+ screens" requirement.
- Dedicated screens keep each feature focused and easier to present.

## 5. Offline-First Data Strategy

Initial content is seeded locally for immediate offline use.

Teacher actions (notes, favorites, schemes) are saved to Room.

Why this step was necessary:

- Teachers may lose connectivity; core usage must continue offline.
- Local persistence improves speed and reliability.

## 6. Online Update Strategy (Legal + Practical)

Instead of embedding copyrighted source content:

- The app syncs metadata and links from `resource\_index.json`.
- It also generates companion links for video lessons and notes searches.

Why this step was necessary:

- Respects copyright and source ownership.
- Still provides timely access to updated official guidance.

## 7. Scheme of Work Module

A dedicated builder allows:

- Week-based planning
- Curriculum-aligned objectives and activities
- Assessment and resources mapping
- Offline save and sharing
- In-app guide explaining how to design a proper scheme

Why this step was necessary:

- This directly answers teacher workflow needs beyond content viewing.
- Converts app value from "reader" to "teaching productivity tool."

## 8. UI/UX and Animation Decisions

Added:

- Splash transition animation
- Animated visibility and content transitions
- Collapsible detail sections

Why this step was necessary:

- Improves user engagement and readability
- Demonstrates modern Compose UI capability as expected in capstone quality standards

## 9. Code Quality and MainActivity Compliance

`MainActivity` is intentionally minimal:

- `setContent { CbcToolkitAppRoot() }`

All business logic and state are outside `MainActivity`.

Previews were added to improve design-time validation.

Why this step was necessary:

- Directly matches the audit requirement in the capstone brief.
- Reduces coupling and accidental logic leakage into Activity.

## 10. Testing and Verification

Executed checks:

- Unit tests (`testDebugUnitTest`)
- APK build (`assembleDebug`)

Tests cover core behavior such as:

- topic observation
- favorites toggling
- note persistence behavior

Why this step was necessary:

- Confirms reliability of key teacher workflows.
- Required by the testing component of the capstone rubric.

## 11. Git Workflow and Incremental Delivery

Work was delivered in small, descriptive commits by feature stage:

- data model
- UI modules
- sync behavior
- tests
- docs and polish

Why this step was necessary:

- Improves traceability and reviewability.
- Aligns with the collaborative git workflow grading criterion.

## 12. Final Functional Summary

The app now supports:

- Full offline usage of seeded CBC topics
- Notes and favorites persistence

- Online resource sync + offline reuse
- Resource detail with links/videos/notes access
- Scheme of Work creation, storage, and sharing
- Animated and structured Compose interface

### **13. Suggested Future Enhancements**

- Background scheduled sync with WorkManager
- Better source ranking for search links
- Scheme export to PDF directly inside app
- Role-based contributor/admin resource management

### **14. Conclusion**

The project moved from concept to a functional, rubric-compliant, offline-first CBC teaching assistant. Each technical step was intentionally chosen to satisfy a user need and a capstone requirement while keeping the app practical for real classroom conditions.